

Priority and Round Robin Scheduling Assignment

Name: Hitesh Sangwan | Roll No: 2401730218

1. Objective & Description

This assignment demonstrates the implementation and evaluation of two fundamental CPU scheduling algorithms in Operating Systems: **Non-Preemptive Priority Scheduling** and **Round Robin Scheduling**.

Key Concepts

- **Priority Scheduling (Non-Preemptive):** Processes are allocated the CPU based on assigned priorities. In this convention, a lower numerical value indicates a higher priority. Once a process starts execution, it runs to completion before yielding the CPU.
- **Round Robin Scheduling:** A preemptive scheduling algorithm designed for time-sharing systems. Each process is assigned a fixed time quantum (e.g., 3 units) in a cyclic order, ensuring fair CPU distribution and minimal response time.

2. Source Code Implementation

```
processes = [  
    {"pid": "P1", "arrival": 0, "burst": 7, "priority": 2},  
    {"pid": "P2", "arrival": 2, "burst": 4, "priority": 1},  
    {"pid": "P3", "arrival": 4, "burst": 1, "priority": 3},  
    {"pid": "P4", "arrival": 5, "burst": 4, "priority": 2},  
]
```

```
PRIORITY_CONVENTION = "Lower number = higher priority"
```

```
TIME_QUANTUM = 3
```

```

def priority_scheduling(processes):
    current_time = 0
    completed = []
    result = []

    while len(completed) < len(processes):
        ready = [p for p in processes if p["arrival"] <= current_time and p["pid"] not in completed]

        if not ready:
            next_time = min(p["arrival"] for p in processes if p["pid"] not in completed)
            result.append({"pid": "Idle", "start": current_time, "end": next_time})
            current_time = next_time
            continue

        p = min(ready, key=lambda x: (x["priority"], x["arrival"], x["pid"]))
        end = current_time + p["burst"]
        result.append({"pid": p["pid"], "start": current_time, "end": end})
        current_time = end
        completed.append(p["pid"])

    return result

```

```

def round_robin(processes, quantum):
    if quantum <= 0:
        raise ValueError("time quantum must be positive")

    remaining = {p["pid"]: p["burst"] for p in processes}
    order = sorted(processes, key=lambda p: p["arrival"])

    current_time = order[0]["arrival"]
    queue = [order[0]["pid"]]
    i = 1
    result = []

    while queue or i < len(order):

```

```
if not queue:
    next_arrival = order[i]["arrival"]
    result.append({"pid": "Idle", "start": current_time, "end": next_arrival})
    current_time = next_arrival
    queue.append(order[i]["pid"])
    i += 1
```

```
pid = queue.pop(0)
run_time = min(quantum, remaining[pid])
start = current_time
current_time += run_time
result.append({"pid": pid, "start": start, "end": current_time})
```

```
remaining[pid] -= run_time
```

```
while i < len(order) and order[i]["arrival"] <= current_time:
    queue.append(order[i]["pid"])
    i += 1
```

```
if remaining[pid] > 0:
    queue.append(pid)
```

```
return result
```

```
def show_result(title, intervals):
```

```
    print("\n" + title)
    print("Process Start End")
```

```
    for slot in intervals:
```

```
        print(f'{slot["pid"]:<9} {slot["start"]:<7} {slot["end"]}')

```

```
print("INPUT PROCESSES")
print("PID AT BT Priority")
```

```

for p in processes:
    print(f'{p["pid"]:<5} {p["arrival"]:<4} {p["burst"]:<4}{p["priority"]}')

print(f"\nPriority convention: {PRIORITY_CONVENTION}")
print(f"Round Robin time quantum: {TIME_QUANTUM}")

show_result("PRIORITY SCHEDULING (non-preemptive)", priority_scheduling(processes))
show_result("ROUND ROBIN SCHEDULING", round_robin(processes, TIME_QUANTUM))

```

3. Execution & Output Analysis

Below is the recorded terminal execution showing the schedule generation for both algorithms:

```

(scienhac@ScienHACDeVIL) - [~/OS-Second-Program]
$ python -u "/home/scienhac/OS-Second-Program/round_robin_priority.py"
INPUT PROCESSES
PID AT BT Priority
P1 0 7 2
P2 2 4 1
P3 4 1 3
P4 5 4 2

Priority convention: Lower number = higher priority
Round Robin time quantum: 3

PRIORITY SCHEDULING (non-preemptive)
Process Start End
P1 0 7
P2 7 11
P4 11 15
P3 15 16

ROUND ROBIN SCHEDULING
Process Start End
P1 0 3
P2 3 6
P1 6 9
P3 9 10
P4 10 13
P2 13 14
P1 14 15
P4 15 16

```